實作報告 · 數位 IC 設計 · EDA

IC 設計的 EDA 是什麼？

從一個 FIFO、一個 16 位元加法器，到一顆 RISC-V SoC：用實際操作走過模擬、驗證、合成、佈局、繞線、時序與位元流，理解 EDA 如何把「想法」變成「可製造的電路」。

操作日期：**2026-10-29**　工具：**veryl2chip IDE**、**hack-computer HDL**

執行環境：**瀏覽器 / WebAssembly**

4 / 4
FIFO 的 RTL、通用閘、iCE40、標準元件四種實作，測試結果一致

48.3 → 493.9
同一設計的 FPGA 與 ASIC Fmax（MHz），相差 10.2 倍

25.9 → 0.70
每週期能量（pJ），ASIC 約為 FPGA 的 1/37

1227 LC
RISC-V SoC 位元流實際佔用的邏輯單元，Fmax 17 MHz

目錄

1 EDA 是什麼：一句話與一個比喻

2 為什麼 IC 設計離不開 EDA

3 IC 設計流程與 EDA 工具鏈

4 本次使用的軟體與操作環境

5 實作一：FIFO 256×8 的完整流程

6 實作二：Inc16 與迷你 EDA

7 實作三：RISC-V CPU 與「放不下」

8 從操作看 EDA 的五個價值

9 EDA 的產業版圖與發展趨勢

10 與 Cornell ECE 5745 課程的對照

11 結論

A 附錄：重現步驟、量測總表、名詞

1 EDA 是什麼：一句話與一個比喻

EDA（Electronic Design Automation，電子設計自動化）是用來設計、驗證、實現與分析晶片（及電路板）的一套軟體工具與方法學。對數位 IC 而言，它最重要的任務，是讓人類寫下的硬體描述（RTL），經過一連串自動化步驟，變成兩種最終產物之一：**FPGA 的位元流（bitstream）**，或 **ASIC 的 GDSII 版圖**。

> **一句話定義**
>
> EDA 是硬體的**編譯器**、**測試框架**、**排版軟體**與**量測儀器**的總和：它把「我想要什麼功能」翻譯成「哪一顆閘放在哪裡、用哪一層金屬連起來」，並在做出來之前告訴你「這樣會不會太慢、太熱、太大、放不下」。

很多人第一次接觸 EDA，會以為它只是「畫電路圖的軟體」或「模擬器」。但真正商用 IC 設計的 EDA 是一條**工具鏈**，每一站都會改變設計的表示法（representation），也每一站都會加入新的物理限制：

軟體開發的類比	IC / FPGA 的對應	轉換了什麼
原始碼	RTL（Veryl / Verilog / VHDL）	人的意圖 → 可被工具解析的時序邏輯描述
編譯器	邏輯合成（logic synthesis）	RTL → 與製程無關的閘級網表（AND / OR / DFF…）
單元測試	testbench、assertion、形式驗證	用測試與數學證明檢查功能是否正確
連結器與最佳化	技術映射（technology mapping）	通用閘 → LUT4、標準元件、BRAM / SRAM
排版引擎	佈局與繞線（placement & routing）	邏輯連接 → 晶片上的實際座標與金屬線
效能分析器	STA、功耗、IR drop、DRC / LVS	「能不能跑」、「會不會燒」、「能不能做」
執行檔	位元流 / GDSII	可載入 FPGA，或可送進晶圓廠製造

這個比喻有一個關鍵差異：**軟體編譯失敗可以重編，晶片做錯通常不能重來。** 一組先進製程光罩動輒數百萬到數千萬美元，一次改版就是幾個月的時間。因此 EDA 不只是「讓設計變快」，更是「讓錯誤在便宜的時候就被抓到」。

2 為什麼 IC 設計離不開 EDA

一顆現代處理器有數百億個電晶體，連線的數量與彼此的時序關係，早已超出人腦可以逐一手算的範圍。EDA 之所以不可取代，可以從四個壓力來看。

▦

規模：數量級問題

一個 256×8 的 FIFO，用通用閘描述就已是 **8,660 個元件**；一顆 RV32I 核心合成後是 **9,151 個閘、5,337 個 LUT4、1,056 個 DFF**。這種規模只能靠工具做階層展開、化簡與對應。

$

成本：錯誤很貴

FPGA 燒錯可以重燒；ASIC 光罩做錯，損失的是整套光罩與數個月產能。EDA 的驗證與簽核（signoff）流程，本質上是在**把錯誤提前**。

⊙

物理：互相拉扯的約束

速度、功耗、面積、良率不是獨立的。把元件排緊可以縮短線長，卻可能造成壅塞與 IR drop；提高時脈可以變快，卻可能讓功耗與時序餘裕惡化。

◈

設計空間：同一份 RTL，多種實現

同一份 FIFO 可以放進 FPGA（1 個 BRAM、60 個 LUT4），也可以做成 ASIC（166 個標準元件、1 個 SRAM）。哪個才對？要靠 EDA 把**PPA（Performance / Power / Area）**量出來再選。

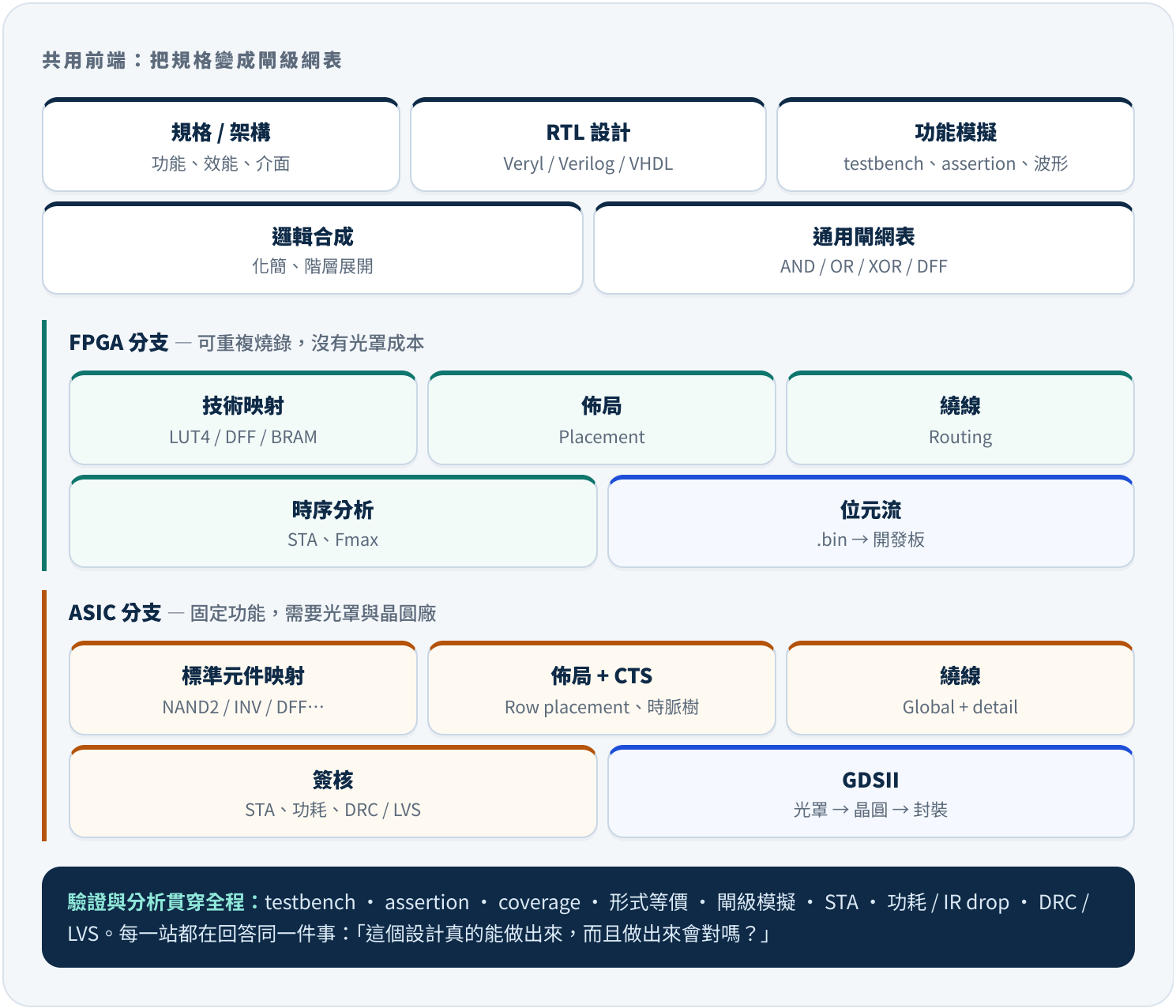
PPA 是什麼

Performance（效能，常用 Fmax 或關鍵路徑延遲表示）、**Power**（功耗／每週期能量）、**Area**（面積）。這三個字幾乎是所有 IC 設計取捨的共同語言；EDA 的工作，就是讓這三個數字在每個階段都「看得見」。

還有一個常被忽略的作用：**EDA 是團隊協作的共同語言**。前端工程師交付 RTL 與 testbench，後端工程師交付佈局、時序與 DRC 報告，製程廠交付 PDK 與標準元件庫；大家靠同一套格式（Verilog、LEF/DEF、SDC、GDSII、SPEF…）溝通。沒有 EDA，這些角色無法交接。

3 IC 設計流程與 EDA 工具鏈

下圖把數位 IC 的典型流程拆成「前端（front-end）」與「後端（back-end）」，並標出 FPGA 與 ASIC 在合成之後的分岔。本報告實際操作的 veryl2chip，就是把這張圖做成可以點選的七個（FPGA）與九個（ASIC）階段。



3.1 前端與後端

前端關心「邏輯對不對」：RTL 撰寫、功能模擬、邏輯合成、形式驗證。輸出的是一份與製程無關（technology-independent）的閘級網表。**後端**關心「物理上做不做得到」：把閘對應到真正的標準元件或 FPGA 資源，安排位置、拉線、插入時脈樹，最後做時序、功耗、DRC / LVS 簽核。

兩者之間不是單向的。後端發現「這條路徑太慢」時，會回頭要求前端改微架構（多加一級 pipeline、拆開組合邏輯），或要求合成工具換一種映射策略。這種來回，就是 EDA 工具與設計者共同進行的**設計空間探索**。

3.2 FPGA 與 ASIC 的差異

面向	FPGA	ASIC
實作方式	可程式化的 LUT、正反器、BRAM、DSP	固定功能的標準元件與 SRAM 巨集，經光罩製造
製造成本	無光罩成本（NRE 低），晶片本身較貴	光罩與設計 NRE 極高，量大時單位成本低
效能 / 功耗	較低 Fmax、較高功耗（可程式開關與繞線多）	較高 Fmax、較低功耗，可做精細的時脈與電源規劃
彈性	可重複燒錄、現場更新	製造後功能固定，改版等於重新流片
典型流程	合成 → 技術映射 → 佈局 → 繞線 → 位元流	合成 → DFT → 佈局 → CTS → 繞線 → 簽核 → GDSII
適合	原型、驗證、少量多樣、教學	大量生產、效能 / 功耗 / 面積敏感的產品

本報告的 FIFO 正好把這個差異量化出來：同一個 RTL、同一組測試，FPGA 版本 Fmax 48.3 MHz、每週期 25.9 pJ；ASIC 版本 Fmax 493.9 MHz、每週期 0.70 pJ。但 ASIC 需要光罩與固定的標準元件庫，FPGA 則可以立刻重燒。這就是 EDA 幫人做的第一種取捨。

4 本次使用的軟體與操作環境

為了不讓報告停留在名詞解釋，我在 2026-10-29 實際使用了兩套瀏覽器版 EDA 環境。所有數字都是操作後從介面讀下來的，不是引用別人的截圖；重現步驟放在附錄 A。

▣

veryl2chip IDE（主要）

核心以 Rust 撰寫、編譯成 WebAssembly 的完整數位 EDA：Veryl HDL 編輯器、事件驅動模擬器、邏輯合成、iCE40 / ECP5 FPGA 流程、ASIC 標準元件流程（含 SKY130）、STA、功耗、虛擬 iCEBreaker。介面支援繁體中文。

開啟位置：ccc-eda.github.io/veryl2chip/web/dist/index.html

▤

hack-computer HDL（對照）

JavaScript 版 Nand2Tetris 工具鏈。HDL 模擬器可載入章節晶片、跑 `.tst`、檢視階層電路圖，並內建一個迷你 EDA：MicroSyn + MicroPnR，示範 FPGA / ASIC 的七個階段與退火佈局。

開啟位置：ccc-js.github.io/hack-computer/dist/hdl.html

操作環境

Windows 11＋Chromium 架構瀏覽器（OpenCode 內嵌，Chrome 152 / Electron 44）。兩套工具都不需要安裝、不需要伺服器；veryl2chip 的模擬器、合成器、佈局繞線器全部以 WebAssembly 在工作執行緒中執行。實際操作包含：按「▶ 測試」、按「✓ 全面驗證」、切換七／九個合成階段、按「⚡ 虛擬板」燒錄，以及讀取狀態列與右側細節面板的數字。

4.1 為什麼選這兩套，而不是直接開商用工具？

商用 ASIC EDA（Synopsys、Cadence、Siemens EDA）需要授權、PDK 與大量環境設定，通常只在學校或公司內部使用；它們的強項是「簽核級」的準確度與完整流程。相對地，veryl2chip 與 hack-computer 把**整條流程攤開在瀏覽器裡**，每個階段都能點進去看閘、LUT 真值表、標準元件、繞線與時序，非常適合理解「EDA 到底在做什麼」。兩者的定位不同：一個是產業的簽核工具，一個是透明可觀察的教學／研究工具。

Cornell ECE 5745 的課程講義則提供了另一個參照：它用 Synopsys / Cadence 教前端與後端，也講合成演算法（Topic 12）與實體設計演算法（Topic 13），並帶學生跑 OpenLANE 開源流程。這正好補上「商用工具怎麼用」與「工具背後演算法是什麼」兩塊。

5 實作一：FIFO 256×8 的完整流程

第一個設計是 256×8 的同步 FIFO，資料在 `i_pop` 的下一拍出現，記憶體以同步讀取推斷成 block RAM。這個設計不大，卻包含了數位 IC 最典型的三件事：**狀態（讀寫指標）、記憶體（RAM）、控制（full / empty）**，很適合走完整條流程。

5.1 設計與 testbench

FIFO 的核心邏輯如下（節錄自 veryl2chip 範例 `fifo.veryl`）：

```
module Fifo #(
    param ADDR_W: u32 = 8,
    param DATA_W: u32 = 8,
) (
    i_clk  : input  clock        ,
    i_rst  : input  reset        ,
    i_push : input  logic        ,
    i_data : input  logic<DATA_W>,
    i_pop  : input  logic        ,
    o_data : output logic<DATA_W>,
    o_empty: output logic        ,
    o_full : output logic        ,
    o_count: output logic<ADDR_W + 1>,
) {
    var mem  : logic<DATA_W> [1 << ADDR_W];
    var wp   : logic<ADDR_W>;
    var rp   : logic<ADDR_W>;
    var count: logic<ADDR_W + 1>;
    var rdata: logic<DATA_W>;

    let do_push: logic = i_push && (count != (1 << ADDR_W));
    let do_pop : logic = i_pop  && (count != 0);

    always_ff (i_clk) {
        if do_push { mem[wp] = i_data; }
        if do_pop  { rdata   = mem[rp]; }
    }

    always_ff {
        if_reset { wp = 0; rp = 0; count = 0; }
        else {
```

```
            if do_push { wp = wp + 1; }
            if do_pop  { rp = rp + 1; }
            if do_push && !do_pop      { count = count + 1; }
            else if do_pop && !do_push { count = count - 1; }
        }
    }

    assign o_data  = rdata;
    assign o_count = count;
    assign o_empty = count == 0;
    assign o_full  = count == (1 << ADDR_W);
}
```

Veryl 用 `always_ff` 表示時序邏輯、`if_reset` 統一處理重設；`=` 在 `always_ff` 內代表「下一拍生效」，等於 Verilog 的 `<=`。

testbench `TestFifo` 依序檢查：重設後為空、push 三個位元組、依序 pop 出 `0x11 / 0x22 / 0x33`、清空後再填滿 256 筆、滿了之後不能再 push。這些 `$assert` 就是 EDA 模擬階段最早、也最便宜的把關。

5.2 功能模擬：0.07 秒的第一次把關

1 testbench 通過	**0.07** s 模擬時間（WebAssembly 背景執行緒）
281 ms 引擎啟動 / 編譯時間	**0 / 0** 錯誤 / 警告

結果顯示 TestFifo ✓ PASS，結束原因為 `$finish`。這個階段只證明「RTL 的行為符合 testbench」，還沒有證明「合成後也一樣」，更沒有證明「放得進晶片、跑得到目標頻率」。

5.3 全面驗證：四種實作，同一個答案

veryl2chip 的「✓ 全面驗證」會把同一個 testbench 跑在四種實作上：原始 RTL、通用閘網表、iCE40 元件網表、ASIC 標準元件網表。這是對合成工具最直接的不信任測試：**如果合成器有 bug，四份結果就會不一致。**

實作	規模（介面顯示）	TestFifo	意義
RTL（Veryl）	1 個 FIFO 模組	✓ PASS	行為基準
通用閘（generic gates）	8,660 個元件	✓ PASS	合成後仍是同一個邏輯
iCE40（LUT4 / DFF / BRAM）	86 個元件	✓ PASS	對應到 FPGA 資源後仍正確
ASIC 標準元件	151 個元件	✓ PASS	對應到標準元件庫後仍正確

量測位置：veryl2chip「測試結果」面板的驗證矩陣；耗時 6.3 秒。註：驗證矩陣的標準元件計數（151）與 ASIC 合成視圖的 166 個標準元件來自不同視圖，計數方式可能含記憶體／邊界元件，本文照實呈現，不強行視為同一數字。

為什麼 8,660 個閘會變成 86 個 iCE40 元件？

因為「閘」和「FPGA 元件」不是同一種東西。通用閘網表把 256×8 的記憶體拆成大量正反器與多工器；iCE40 映射時，整塊記憶體被收進 **1 個 SB_RAM40_4K**，其餘控制邏輯才用 LUT4 與 DFF 實現。**元件數不能跨技術直接比較**，必須在同一個製程／元件庫裡看 PPA。

5.4 FPGA 實現：60 個 LUT4、1 個 BRAM

切到「FPGA → ⑤ Placement 退火」，工具完成技術映射、佈局與繞線，狀態列顯示：

60
LUT4（4 輸入查表）

25
DFF（正反器）

48.3 MHz
Fmax（關鍵路徑 20.70 ns）

1.2 %
iCE40UP5K 的 5,280 LC 中使用 61 LC

右側細節面板同時給出：1 個 `SB_RAM40_4K`、632 個可程式繞線開關（pip）、25.9 pJ/週期、在 48.3 MHz 下約 1.340 mW（靜態 0.090 mW）。FPGA 的面積是「固定晶片」，設計只佔 61 個 LC，所以真正的成本是「這顆 FPGA 多少錢」，而不是「這 61 個 LC 多少面積」。

5.5 ASIC 實現：166 個標準元件、493.9 MHz

切到「ASIC ・ vlib → ⑨ Power + IR Drop」，同一份 RTL 被映射到自製的 vlib 標準元件庫（約 130 nm 等級的教學模型），再經過列式佈局、時脈樹、繞線、STA 與功耗分析：

166
標準元件（含 25 個 DFF）

493.9 MHz
Fmax（關鍵路徑 2.02 ns）

7,828 μm^2
元件面積；晶粒 98×96 µm，利用率 82%

0.70 pJ
每週期能量（@ 48.3 MHz 比較基準）

繞線結果是 7.39 mm 金屬線、1,804 個 via；在 48.3 MHz 下功耗約 0.0341 mW（漏電 0.0003 mW）。ASIC 的「面積」才是真正的矽面積，所以 7,828 μm^2 這個數字會直接影響每顆晶片的成本。

能量花在哪裡？

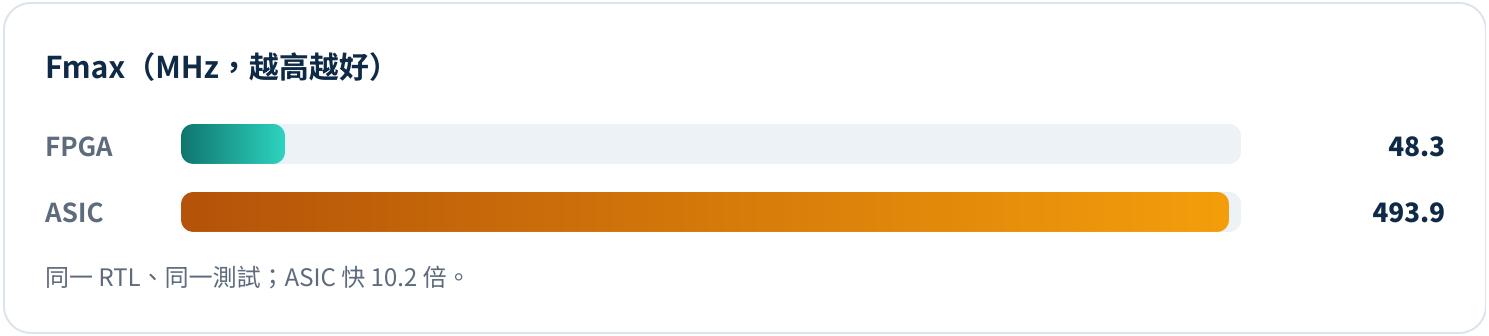
ASIC 的每週期能量分解（畫面數值，總和約 0.70 pJ）很值得注意：

來源	每週期能量	占比	說明
訊號 net	0.282 pJ	40.3%	元件之間的金屬線充放電
時脈樹	0.258 pJ	36.9%	時脈要驅動所有正反器
元件內部	0.149 pJ	21.3%	閘本身的切換
SRAM	0.010 pJ	1.4%	記憶體存取

0.282 + 0.258 + 0.149 + 0.010 = 0.699 pJ，與顯示的 0.70 pJ 相符。這說明在深次微米設計裡，**「線」與「時脈」往往比「閘」更耗能**；這也是為什麼後端工程師要花大量時間做 placement、CTS 與繞線最佳化。

5.6 FPGA vs ASIC：把取捨變成數字

指標	FPGA（iCE40UP5K）	ASIC（vlib 標準元件）
實作	LUT4 + DFF + BRAM（可重燒）	標準元件 + SRAM 巨集（需流片）
邏輯資源	61 LC（1.2%）、25 DFF、0 CARRY	166 個元件、25 DFF
記憶體	1 × SB_RAM40_4K	1 × SRAM 巨集
面積	固定晶片（5,280 LC / 30 BRAM）	元件 7,828 μm²；晶粒 98×96 μm；利用率 82%
繞線	632 個 pip（可程式開關）	7.39 mm 金屬線、1,804 個 via
關鍵路徑	20.70 ns	2.02 ns
Fmax	48.3 MHz	493.9 MHz（×10.2）
每週期能量	25.9 pJ	0.70 pJ（÷37）
功耗 @ 48.3 MHz	1.340 mW（靜態 0.090）	0.0341 mW（漏電 0.0003）



同一 RTL、同一測試；ASIC 快 10.2 倍。



ASIC 約為 FPGA 的 1/37；可程式繞線與較大的開關電容是主因。



資料來源：veryl2chip「⇄ FPGA vs ASIC 比較」頁面；測試日期 2026-10-29。ASIC 使用 vlib 教學用標準元件庫，數字是此流程模型的估計，不是商業 signoff 或實際晶片量測。

5.7 這一輪學到什麼

同一份 125 行的 FIFO，在四個抽象層級都通過同一個 testbench，卻有完全不同的成本結構：通用閘 8,660 個元件、FPGA 86 個元件（1 個 BRAM）、ASIC 166 個元件（1 個 SRAM）。**EDA 的價值不是「把電路畫出來」，而是讓這些差異可以被量測、被比較、被選擇。** 如果沒有合成與佈局繞線，設計者只知道「邏輯對」，不知道「這樣做值不值得」。這正是 EDA 從「輔助繪圖」升級為「設計方法」的關鍵。

6 實作二：Inc16 與迷你 EDA

第二個設計回到 Nand2Tetris 的教學語境：用 `hdl.html` 模擬一個 16 位元遞增器 `Inc16`，再按「合成 LUT」看它如何被映射到 FPGA 與 ASIC。這個例子小到可以完全看懂，正好用來拆解「合成」到底做了什麼。

6.1 HDL 模擬與階層電路圖

`Inc16` 的 HDL 只有一行真正的工作：把 `Add16` 的第一個加數接 `in`，第二個加數的第 0 位接 `true`、其餘接 `false`。

```
CHIP Inc16 {
    IN in[16];
    OUT out[16];

    PARTS:
    Add16(a=in, b[0]=true, b[1..15]=false, out=out);
}
```

按「執行」後，`.tst` 的輸出表全部符合 `.cmp`，顯示 ✓通過：

in（二進位）	out（二進位）	十進位意義
`0000 0000 0000 0000`	`0000 0000 0000 0001`	0 → 1
`1111 1111 1111 1111`	`0000 0000 0000 0000`	65535 → 0（溢位）
`0000 0000 0000 0101`	`0000 0000 0000 0110`	5 → 6
`1111 1111 1111 1011`	`1111 1111 1111 1100`	65531 → 65532

按「檢視電路圖」可以看到階層：`Inc16` 內含一個 `Add16`，`b[0]` 綁 `true`、`b[1..15]` 綁 `false`。這張圖可以一路點進 `Add16`、`FullAdder`，直到最底層的 `Nand` 與 `DFF`。**「階層」是硬體設計控制複雜度的第一個工具**：你不需要一次看到 224 個閘，只需要看到一顆 `Add16`。

6.2 迷你 EDA：224 個閘 → 60 個 LUT4

按「合成 LUT」後，畫面顯示七個階段：① RTL→Logic、② Logic Netlist、③ Tech-indep Syn、④ LUT4 Techmap、⑤ Placement、⑥ Routing、⑦ STA，並可切換 FPGA / ASIC 後端。這次的結果：

60
LUT4（FPGA 技術映射後）

0
DFF（純組合邏輯）

23.0 MHz
Fmax（關鍵路徑 43.50 ns）

10×10
佈局格子；利用率 0.60

其他數字：I/O Pad 32 個、總線長 78（畫面單位）、原始閘數 224。若切到 ASIC 視圖，同一份設計變成 **174 個標準元件**（NAND2×65、AOI21×16、INV×78、MUX2×15），面積 319.2 μm²、66 列、線長 1,705.3 μm、Fmax 約 81 MHz。

階段	FPGA 視圖	ASIC 視圖	在做什麼
① RTL→Logic	展開 `Inc16` → `Add16` → 全加器	同左	解析 HDL、建階層、攤平實例
② Logic Netlist	224 個閘	224 個閘	每個行為變成 AND / XOR / NOT / DFF
③ Tech-indep Syn	化簡、常數傳播	同左	去掉重複與沒用到的邏輯
④ 技術映射	60 個 LUT4 + 0 DFF	174 個標準元件	對應到真正的製程元件
⑤ 佈局	10×10 格子、利用率 0.60	66 列	決定元件位置，縮短連線
⑥ 繞線	總線長 78	線長 1,705.3 µm	用可程式開關或金屬層連接
⑦ 時序	43.50 ns → 23.0 MHz	約 81 MHz	找關鍵路徑，算 Fmax

資料來源：hack-computer `hdl.html` 的「合成 LUT」浮層，2026-10-29。此為教學用的簡化模型（MicroSyn / MicroPnR），數字用於理解流程，不應與商用 signoff 直接比較。

為什麼 LUT4 的數量不是「224 ÷ 4」？

因為一個 LUT4 不是「4 個閘」，而是**一張 16 位元的真值表**：任何 4 個輸入的布林函數都可以用它實現。合成器會把邏輯切成 4 輸入的切集（cut），再把每個切集的真值表燒進 LUT。因此 224 個閘可能變成 60 個 LUT4，也可能更多或更少，取決於電路結構與化簡。ASIC 則沒有 LUT，必須用 NAND2、INV、AOI21 等標準元件重新組出同樣的函數，所以又是另一組數字。

6.3 這個例子的意義

`Inc16` 只是一個 16 位元加一，卻已經完整展示了 EDA 的核心迴圈：**描述（HDL）→ 模擬（.tst）→ 合成（閘）→ 映射（LUT / 標準元件）→ 佈局繞線（座標）→ 時序（Fmax）**。對初學者來說，這比直接看一顆 CPU 更容易建立心智模型；對有經驗的設計者來說，它提醒你：每一個階段都可能改變數字，但**行為必須保持一致**。

7 實作三：RISC-V CPU 與「放不下」

第三個設計把規模拉大：veryl2chip 內建的單週期 RV32I 核心 `RiscvCore`，以及加上程式 ROM、資料 RAM、LED 的 `RiscvSoc`。這一輪最重要的收穫不是「跑得多快」，而是**EDA 如何在編譯期就告訴你「這顆晶片放不下」**。

7.1 單週期 RV32I：功能對，但放不進 FPGA

選擇「RV32I 單週期 CPU」範例，直接跳到 FPGA 的 ⑤ Placement。合成結果是 **9,175 → 9,151 個閘、5,337 個 LUT4、1,056 個 DFF**；接著佈局繞線回報：

> 佈局繞線失敗：`design does not fit: needs 5441 LCs (the device has 5280) and 167 IO pins (the package has 39)`

iCE40UP5K-SG48 只有 5,280 個 LC、39 個 I/O 腳。這個核心的暫存器檔用正反器實現，再加上單週期 CPU 的資料路徑，結果**超出 161 個 LC，且 I/O 需求遠超封裝腳數**。工具沒有把錯誤留到燒錄或上電才發生，而是在佈局階段就明確拒絕，並建議改用「RISC-V SoC（FPGA）」範例或 ASIC 流程。

這一幕就是 EDA 的價值

「放不下」不是設計者的疏忽，而是**架構與資源的衝突**。EDA 把這個衝突量化成兩個數字（5,441 > 5,280、167 > 39），讓設計者可以在**改架構**（多週期、把暫存器／記憶體放進 BRAM）、**換晶片**（更大的 FPGA）、或**改流程**（走 ASIC）之間做選擇。如果沒有這一關，這些數字要等到板子回來才會變成除錯成本。

7.2 RISC-V SoC：把記憶體放進 BRAM 就放得下

改用 multi-cycle 的 `RiscvSoc`：暫存器檔與記憶體改用同步讀取的 RAM，推斷成 block RAM；程式 ROM 也放在 BRAM。結果大幅收斂：

2,463	1,164
合成後閘數（原始 11,969）	LUT4

232
DFF

17.0 MHz
Fmax（板子時脈 12 MHz，有餘裕）

兩個 testbench 都通過：`TestRiscvCore` 與 `TestRiscvSoc`，各以 `$finish` 結束，總計 0.19 秒。按「⚡ 虛擬板」後，工具合成、佈局繞線、產生位元流並燒錄，回報：

```
RiscvSoc  位元流：1227 LC，Fmax 17.0 MHz ≥ 12 MHz ✓
接線：CLK↔i_clk  BTN_N↔i_rst  P1A1↔o_led[0] … P1A10↔o_led[7]
```

虛擬板上的 8 顆 LED 最後顯示為 **1 1 1 0 1 1 0 0**（LED0→LED7），也就是二進位 `0011 0111` = `0x37` = **55**，正好是測試程式計算的 Fibonacci(10)。



LED0 是最低位元。由左至右為 LED0..LED7：1,1,1,0,1,1,0,0 → 0b00110111 = 55。

7.3 從「放不下」到「跑起來」：架構決策被 EDA 推動

這一輪完整展示了 EDA 如何參與**架構決策**，而不只是被動實現：

- **單週期 → 多週期**：減少每個時脈的組合邏輯與資料路徑，換取較低的資源與較高的 Fmax 餘裕。
- **暫存器 → BRAM**：把 32×32 的暫存器檔與記憶體放進 block RAM，LUT4 用量從 5,337 降到 1,164。
- **程式 ROM 常數化**：合成器把只被這個程式用到的硬體移除（未用的暫存器、移位器、byte load 等），所以 SoC 的閘數遠小於完整核心。
- **位元流驗證**：Fmax 17.0 MHz ≥ 12 MHz，才敢說「這顆設計可以在這塊板子上跑」。

這也解釋了為什麼 IC 設計不是「寫完 RTL 就結束」：**RTL 只是提案，EDA 才是把提案變成可執行承諾的過程。**

8 從操作看 EDA 的五個價值

把三次操作放在一起看，EDA 的價值可以歸納成五點。每一點都對應到本報告量到的具體數字。

8.1 正確性：四種實作，一個答案

FIFO 的「全面驗證」把同一個 testbench 跑在 RTL、通用閘、iCE40、標準元件四種實作上，結果 4/4 一致。這個矩陣的意義是：**合成器也是軟體，也會有 bug**；用同一組測試去驗證合成前後，是最實際的交叉檢查。更嚴謹的流程還會做形式等價檢查（formal equivalence），用數學證明兩份網表在所有輸入下行為相同。

8.2 可實現性：在編譯期就說「放不下」

RV32I 單週期核心需要 5,441 LC，但 iCE40UP5K 只有 5,280 LC；需要 167 個 I/O，但封裝只有 39 腳。EDA 在佈局階段就給出明確的失敗訊息，還附上建議。這種「早期失敗」比「晚期失敗」便宜幾個數量級：在 FPGA 上，它只損失幾分鐘；在 ASIC 上，它可能避免一次數百萬美元的光罩錯誤。

8.3 量化：把 PPA 取捨變成表格

FIFO 的 FPGA 與 ASIC 比較，把「哪個比較好」拆成 9 個指標：邏輯資源、記憶體、面積、繞線、關鍵路徑、Fmax、每週期能量、功耗。沒有 EDA，這些只能靠猜；有了 EDA，設計者可以說「我需要 48 MHz、低功耗、可重燒，所以選 FPGA」或「我要 500 MHz、量產一億顆，所以走 ASIC」。**工程決策的前提是可比較的數字。**

8.4 生產力：瀏覽器裡的分鐘級迭代

這次的迭代速度：FIFO 模擬 0.07 秒、四種實作全面驗證 6.3 秒、RISC-V SoC 的佈局繞線約 40 秒、虛擬板燒錄與開機在分鐘內完成。這些數字在 20 年前需要工作站與授權排隊，現在只要一個瀏覽器分頁。WebAssembly 與開源工具鏈把 EDA 的**入門門檻**降到「打開網址就能用」，這對教學、原型與小團隊特別重要。

8.5 透明性：每一個階段都可以點進去看

veryl2chip 與 hack-computer 都把中間結果畫出來：LUT 的真值表、標準元件的位置、繞線的 net、關鍵路徑的每一段延遲、退火佈局的過程。商用工具當然也能產生報告，但通常藏在指令與 log 裡；教學工具把「黑盒子」打開，讓學習者看見**演算法如何改變結果**。例如 Inc16 的 224 個閘變成 60 個 LUT4，或 FIFO 的 8,660 個通用閘變成 86 個 iCE40 元件，這些「數量級跳躍」只有在中間階段才看得懂。

也要知道限制

本報告的 ASIC 數字來自 vlib 教學模型（veryl2chip 另支援 SKY130 開放 PDK），不是商業 signoff；hack-computer 的 MicroSyn / MicroPnR 也是簡化版。真實 tapeout 需要 foundry 的 PDK、Liberty / SPEF / SDC、商用 STA 與 DRC / LVS 工具，以及 silicon validation。把這些數字當成「理解流程與趨勢」的工具，而不是「可以直接下線」的保證。

9 EDA 的產業版圖與發展趨勢

9.1 商用與開源

類別	代表工具 / 公司	角色
商用三大 EDA	Synopsys、Cadence、Siemens EDA（原 Mentor）	前端合成、模擬、形式驗證、後端佈局繞線、STA、DFT、PCB / 封裝
物理分析 / 多物理	Ansys、Keysight、Silvaco	熱、電磁、電源完整性、TCAD、元件模擬
開源前端	Verilator、Icarus Verilog、Yosys、ghdl	模擬、合成、形式 / 等價檢查
開源後端	nextpnr、OpenROAD、OpenLANE、Magic、KLayout	FPGA / ASIC 佈局繞線、DRC / LVS、GDSII
開源 PDK	SkyWater SKY130、GF180MCU	讓學界與小團隊也能跑完整 ASIC 流程
瀏覽器 / 教學	veryl2chip、hack-computer、Wokwi（部分）	免安裝、視覺化、適合課程與原型

9.2 四個明顯趨勢

① 雲端與瀏覽器化

把模擬器、合成器、佈局繞線器編譯成 WebAssembly，讓 EDA 從「工作站授權」變成「開網址就能跑」。veryl2chip 就是這個方向的極端例子：整條流程在前端執行，沒有伺服器。

② 開源流程成熟

Yosys + nextpnr + OpenROAD + SKY130 已經可以做出可運作的晶片；Cornell ECE 5745 也把 OpenLANE 放進課程。開源不是要取代商用 signoff，而是降低學習與原型門檻。

③ AI 輔助設計

機器學習被用在佈局預測、繞線壅塞預測、設計空間探索、測試生成與驗證覆蓋率。大型語言模型也開始協助產生 RTL 與 testbench，但**驗證與簽核仍然是人與工具的責任**。

④ Chiplet 與先進封裝

當單晶片越來越難做，設計拆成多顆 chiplet，EDA 要處理 die-to-die 介面、封裝繞線、熱與電源整合。流程從「一顆晶片」變成「一個系統」。

9.3 對台灣的意義

台灣是全球晶圓代工與 IC 設計的重鎮，EDA 是這個生態系的「共同語言」：設計公司用 EDA 把產品做出來，晶圓廠用 EDA / PDK 確保設計可以製造，封裝測試也需要對應的工具鏈。對學習者而言，理解 EDA 不只是會操作一套軟體，而是理解**從規格、RTL、驗證、佈局到量產的整條價值鏈**；這也是為什麼數位邏輯、計算機組織、VLSI 與 EDA 課程通常會互相搭配。

10 與 Cornell ECE 5745 課程的對照

Cornell 的 **ECE 5745: Complex Digital ASIC Design** 是理解「產業級 EDA 怎麼教」的好參照。課程講義把主題拆成 HDL、CMOS 元件與電路、全客戶設計、自動化方法、ASIC 與 custom 的差距、封裝／電源／時脈／

I/O、測試與驗證、合成演算法、實體設計演算法等；實作則包含前端、後端、自動化流程、TinyRV2 加速器、SRAM 產生器與 OpenLANE 流程。

ECE 5745 的主題	本報告的對應操作	補充說明
Topic 1：Hardware Description Languages	Veryl / Nand2Tetris HDL 撰寫與模擬	RTL 是設計的「原始碼」，語意與時序必須精確
Topic 5：Automated Design Methodologies	FIFO / Inc16 / RISC-V SoC 的合成與實現	自動化把設計者從手繪閘級電路中解放
Topic 8：Testing and Verification	testbench、`$assert`、四種實作全面驗證	驗證不是「跑一次」，而是交叉檢查與覆蓋率
Topic 12：Synthesis Algorithms	224 閘 → 60 LUT4、8,660 閘 → 86 元件	化簡、切集、技術映射決定最終 PPA
Topic 13：Physical Design Automation	Placement、Routing、CTS、STA、功耗 / IR	物理限制讓「邏輯對」變成「做出來也對」
Section 1–3：Front-end / Back-end / Flow	veryl2chip 的七／九階段流程	前端與後端不是先後，而是反覆迭代
Tutorial 4：Synopsys / Cadence ASIC Tools	本報告用開源／教學工具替代	商用工具做 signoff，教學工具做理解

兩者最大的差異在**準確度與規模**：ECE 5745 用商用工具與真實 PDK，處理的是可以下線的設計；本報告用瀏覽器工具，處理的是可以理解的設計。但兩者要建立的直覺是一樣的：**設計者必須知道每個階段在做什麼、會引入什麼限制、以及如何用數字判斷好壞。** 只會按「合成」按鈕而不懂背後演算法，遇到 timing violation 或壅塞時就無法除錯；只懂演算法而不會操作工具，也無法把設計帶到 tapeout。EDA 的教育價值，正在這兩者的交會處。

11 結論

EDA 是 IC 設計的「實現層」。 它接收人類寫下的 RTL，經過模擬、合成、映射、佈局、繞線、時序與功耗分析，最後輸出位元流或 GDSII。它不只是畫圖或模擬，而是一套把「功能意圖」轉換成「物理現實」的工具鏈

與方法學。

這次的實作讓我印象最深的是**抽象層級的落差**：一個 256×8 的 FIFO，在通用閘是 8,660 個元件，在 FPGA 是 86 個元件（1 個 BRAM），在 ASIC 是 166 個標準元件（1 個 SRAM）。同一個行為，不同的實現，Fmax 從 48.3 MHz 跳到 493.9 MHz，每週期能量從 25.9 pJ 降到 0.70 pJ。這些數字不是課本上的形容詞，而是按了幾下滑鼠、跑了幾十秒之後，工具根據真實的元件庫與繞線算出來的。

另一個印象是**失敗的價值**：單週期 RV32I 核心因為 5,441 > 5,280 個 LC、167 > 39 個 I/O 而「放不下」。這個失敗不是挫折，而是 EDA 最有用的功能之一：它讓設計者在花錢之前就知道架構與資源的衝突，然後用多週期、BRAM、換晶片或改流程來解決。最後 RISC-V SoC 以 1,164 個 LUT4、17.0 MHz 的位元流在虛擬板上亮出 LED = 55，完成了從 RTL 到實際運行的閉環。

所以，**IC 設計的 EDA 是什麼？** 它是把「我想要一顆會算 Fibonacci 的晶片」變成「這顆晶片在 17 MHz 下、用 1,227 個邏輯單元、可以燒進這塊板子」的那一整套工具、資料與方法。沒有它，現代晶片的複雜度無法被管理；有了它，設計者才能在速度、功耗、面積與成本之間，做出有根據的選擇。

> **給第一次接觸 EDA 的人**
>
> 不要從「記住所有工具名稱」開始。先從一個小設計（例如 FIFO 或 Inc16）走一次完整流程：寫 RTL → 跑 testbench → 看波形 → 合成 → 看閘／LUT → 佈局繞線 → 看 Fmax。當你看到同一份程式碼變成不同的元件、不同的數字，你就會真正理解 EDA 在做什麼。

A 附錄：重現步驟、量測總表、名詞

A.1 重現步驟

1. **FIFO**：開啟 veryl2chip IDE（#ex=fifo） → 按「▶ 測試」→ 按「✓ 全面驗證」→ 在「看電路」選 `FPGA` 與「⑤ Placement 退火」→ 再選 `ASIC · vlib` 與「⑨ Power + IR Drop」→ 最後選「⇄ FPGA vs ASIC 比較」。
2. **Inc16**：開啟 hack-computer HDL → 章節選「02 加法器/ALU」→ 晶片選 `../02/Inc16` → 按「執行」→ 按「檢視電路圖」→ 按「合成 LUT」→ 切換 FPGA / ASIC 視圖。
3. **RISC-V**：開啟 RV32I 單週期 CPU（#ex=rv32i&stage=place），觀察「放不下」訊息；再開 RISC-V SoC（#ex=soc&stage=place） → 按「▶ 測試」→ 按「⚡ 虛擬板」→ 看 LED。

4. **命令列（可選）**：veryl2chip 也提供 `v2c`：`cargo install veryl2chip-sim veryl2chip-eda`，即可用 `v2c sim / synth / fpga / asic / gatesim` 重跑類似流程。

A.2 量測總表

項目	數值	來源 / 備註
FIFO 模擬	1 個 testbench PASS，0.07 s，0 錯誤 / 0 警告	「測試結果」面板
FIFO 全面驗證	1 個 testbench × 4 種實作全部一致，6.3 s	「✓ 全面驗證」
FIFO 通用閘 / iCE40 / 標準元件	8,660 / 86 / 151 個元件	驗證矩陣
FIFO FPGA	60 LUT4、25 DFF、61 LC（1.2%）、1 BRAM、632 pips、20.70 ns、48.3 MHz	FPGA ⑤；狀態列
FIFO ASIC	166 cells、25 DFF、7,828 μm²、98×96 μm、82%、7.39 mm、1,804 via、2.02 ns、493.9 MHz	ASIC ⑨；比較頁
FIFO 能量 / 功耗	FPGA 25.9 pJ、1.340 mW；ASIC 0.70 pJ、0.0341 mW（@48.3 MHz）	比較頁
Inc16 HDL	4 組測資全部通過（0→1、65535→0、5→6、65531→65532）	hack-computer「執行」
Inc16 FPGA	224 閘 → 60 LUT4 + 0 DFF、10×10、util 0.60、32 I/O、43.50 ns、23.0 MHz	「合成 LUT」FPGA
Inc16 ASIC	174 cells（NAND2 65 / AOI21 16 / INV 78 / MUX2 15）、319.2 μm²、66 列、1,705.3 μm、約 81 MHz	「合成 LUT」ASIC
RV32I 單週期	9,175→9,151 閘、5,337 LUT4、1,056 DFF；需要 5,441 LC / 167 IO，放不下	FPGA ⑤ 失敗訊息
RISC-V SoC	11,969→2,463 閘、1,164 LUT4、232 DFF、17.0 MHz；2 個 testbench PASS	FPGA ⑤；測試結果
RISC-V SoC 位元流	1,227 LC、Fmax 17.0 MHz ≥ 12 MHz；LED = 0b00110111 = 55	虛擬板

A.3 名詞對照

名詞	意思
RTL	Register-Transfer Level，暫存器轉移層級；用時脈與暫存器描述電路行為，是合成工具的輸入。
Testbench	用來測試設計的「測試電路」：產生時脈、餵輸入、檢查輸出。
Synthesis	邏輯合成：把 RTL 轉成閘級網表，並做化簡與最佳化。
Technology mapping	技術映射：把通用閘對應到特定製程的 LUT、標準元件或記憶體。
Placement / Routing	佈局／繞線：決定元件位置，再用金屬線或可程式開關連接。
CTS	Clock Tree Synthesis，時脈樹合成：把時脈均勻地送到所有正反器，控制 skew。
STA	Static Timing Analysis，靜態時序分析：不跑模擬，用延遲模型檢查 setup / hold 是否滿足。
PPA	Performance、Power、Area：效能、功耗、面積，IC 設計的三大取捨。
DRC / LVS	Design Rule Check / Layout Versus Schematic：檢查版圖是否符合製程規則、是否與電路一致。
PDK	Process Design Kit：製程廠提供的設計套件（元件模型、規則、Liberty 等）。
DFT / ATPG	Design for Test / Automatic Test Pattern Generation：讓晶片可被測試，並自動產生測試向量。
GDSII	積體電路版圖的交換格式，是送給晶圓廠的「施工圖」。
Bitstream	FPGA 的設定檔：每個 LUT 的真值表與每個繞線開關的開關狀態。
LUT4	4 輸入查表：16 位元真值表，可實現任意 4 輸入布林函數。
DFF	D 型正反器：時脈邊緣觸發的 1 位元儲存元件。
BRAM / SRAM	FPGA 的塊狀記憶體 / ASIC 的靜態隨機存取記憶體巨集。

名詞	意思
Tapeout	送片：把 GDSII 送進晶圓廠製作光罩並生產。
NRE	Non-Recurring Engineering，一次性工程成本（光罩、設計、驗證）。

A.4 參考資料

1. veryl2chip IDE（線上版）。`https://ccc-eda.github.io/veryl2chip/web/dist/index.html`（2026-10-29 操作）
2. veryl2chip GitHub：README、tutorial、veryl-eda 文件。`https://github.com/ccc-eda/veryl2chip/`
3. veryl2chip 教學：從零開始用 Veryl 設計數位電路。`https://github.com/ccc-eda/veryl2chip/blob/main/tutorial/README.tw.md`
4. hack-computer HDL 模擬器。`https://ccc-js.github.io/hack-computer/dist/hdl.html`（2026-10-29 操作）
5. hack-computer GitHub：Nand2Tetris 全工具鏈＋RISC-V 後端。`https://github.com/ccc-js/hack-computer`
6. Cornell ECE 5745: Complex Digital ASIC Design — Handouts。`https://www.csl.cornell.edu/courses/ece5745/handouts.html`
7. Cornell ECE 5745：ASIC Tool Front-End / Back-End / Automated Flow 課程單元。`https://cornell-ece5745.github.io/`
8. Veryl 官方網站。`https://veryl-lang.org/`
9. Yosys / nextpnr / OpenROAD / OpenLANE 開源工具鏈。`https://yosyshq.net/yosys/`、`https://www.openroad.net/`